

Configuration Web Services for .NET Framework

Developer's Guide

August 2023

This guide describes how to create a client for the Configuration Web services with the .NET framework and C# language.

Five9 and the Five9 logo are registered trademarks of Five9 and its subsidiaries in the United States and other countries. Other marks and brands may be claimed as the property of others. The product plans, specifications, and descriptions herein are provided for information only and subject to change without notice, and are provided without warranty of any kind, express or implied. Copyright © 2023 Five9, Inc.

About Five9

Five9 is the leading provider of cloud contact center software, bringing the power of the cloud to thousands of customers and facilitating more than three billion customer interactions annually. Since 2001, Five9 has led the cloud revolution in contact centers, delivering software to help organizations of every size transition from premise-based software to the cloud. With its extensive expertise, technology, and ecosystem of partners, Five9 delivers secure, reliable, scalable cloud contact center software to help businesses create exceptional customer experiences, increase agent productivity and deliver tangible results. For more information visit www.five9.com.

Trademarks

Five9®

Five9 Logo

Five9® SoCoCare™

Five9® Connect™

Contents

What's New	4
Configuration Web Services for .NET Framework	5
Audience	5
Web Services Platform	5
Creating a .NET Application	6
.NET Core	6
.NET Framework	9
Creating Code	11
Enabling HTTP Basic Authentication	11
C#	12
VB.NET	13
Using Generated Code	14
Source Code	15
AuthHeaderInserter.cs	15
AuthHeaderBehavior.cs	17
Program.cs	18
asyncAddRecordsToList.cs	20
AuthHeaderInserter.vb	23
AuthHeaderBehavior.vb	25
Program.vb	26
CreateUser.vb	28
Resolving Issues in the Reference.vb File	31

What's New

This table lists the changes made in previous releases of this document:

Release	Change	Topic
Aug 2023	Added a note about including the corresponding specified parameter.	CreateUser.vb
Jul 2020	Added a note about using the data center that applies to you.	Web Services Platform
	Added topic:	Resolving Issues in the Reference.vb File
	Updated instructions.	Creating a .NET Application
	Updated examples and code.	Creating Code Source Code
Oct 2014	Added a note about the examples in the guide.	Configuration Web Services for .NET Framework
	Updated topic:	Creating a .NET Application
	Added a note about finding additional code examples on GitHub.	
Sep 2013	Re-branded the guide with the new Five9 logo.	
	Added an example.	End Property

Configuration Web Services for .NET Framework

This guide describes how to create a client for the Configuration Web Services with the .NET framework version 4.6.1 and newer (Visual Studio 2017 and 2019).

Important

The code snippets in this guide are examples only. You cannot compile the snippets because they are incomplete.

[Audience](#)
[Web Services Platform](#)
[Creating a .NET Application](#)

Audience

This guide is intended for developers who want to create contact-center applications. Developers must understand these technologies:

- Client-server architecture and Web Services
- SOAP, HTTP, and XML
- .NET
- C# and Visual Basic
- Overall call-center integration and configuration

Web Services Platform

The Web Services API contains the XML-encoded SOAP methods required to communicate with your client application. The WSDL file is located at this URL:

```
https://api.five9.com/wsadmin[/<version>]/AdminWebService?wsdl&user  
<Five9username>
```

`version`: your version of the WSDL: `v{major}_{minor}`, such as, “v11_0” or “v11_5”.
Five9username: user name of the administrator.

Important

Use the data center that applies to you: `api.five9.com` or `api.five9.eu`.

To ensure that connections are secure, send all requests by Transport Layer Security protocol (HTTPS) or VPN (IPSec or SSH) to this URL:

`https://api.five9.com/wsadmin[/<version>]/AdminWebService`

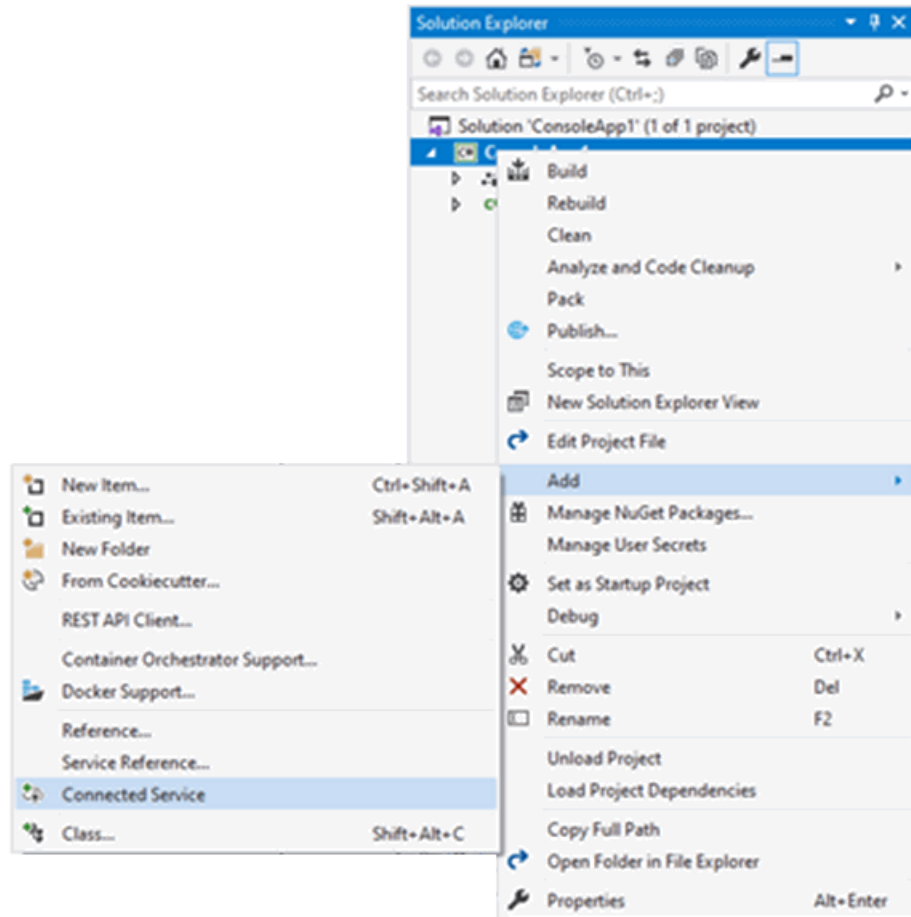
Creating a .NET Application

Visual Studio supports a number of project types. Most support adding a service reference, but the service reference implementation process may vary. This document provides a couple of implementations for adding a service reference to your project within Visual Studio.

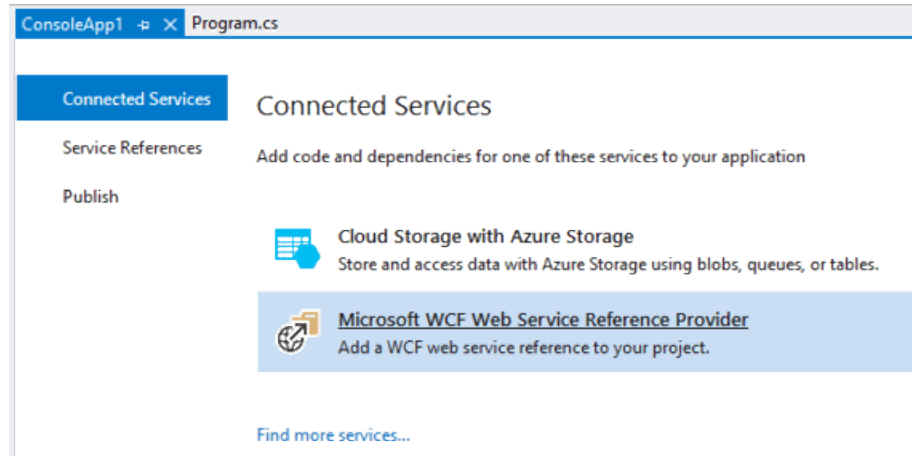
.NET Core

This procedure provides instructions for using Visual Studio to create a .NET Core application and add a reference to the Configuration web service.

- 1 Right-click the project in the **Solution Explorer**.

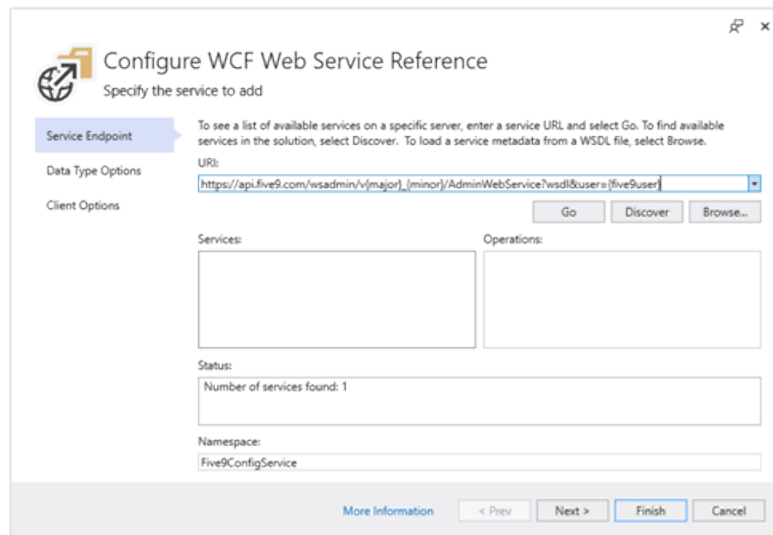


- 2 Select **Add > Connected Service**.
- 3 Select **Microsoft WCF Web Service Reference Provider** in the **Connected Services** pop-up window.



4 Enter the URI for the WSDL.

```
https://api.five9.com/wsadmin/v<major>_  
<minor>/AdminWebService?wsdl&user=<five9user>
```



5 Click **Go**.

The WSDL appears in the **Services** area.

6 Enter `Five9ConfigService` in the **Namespace** field.

The namespace is used in your code to reference the Five9 Configuration Web Service (`WsAdminService`) object.

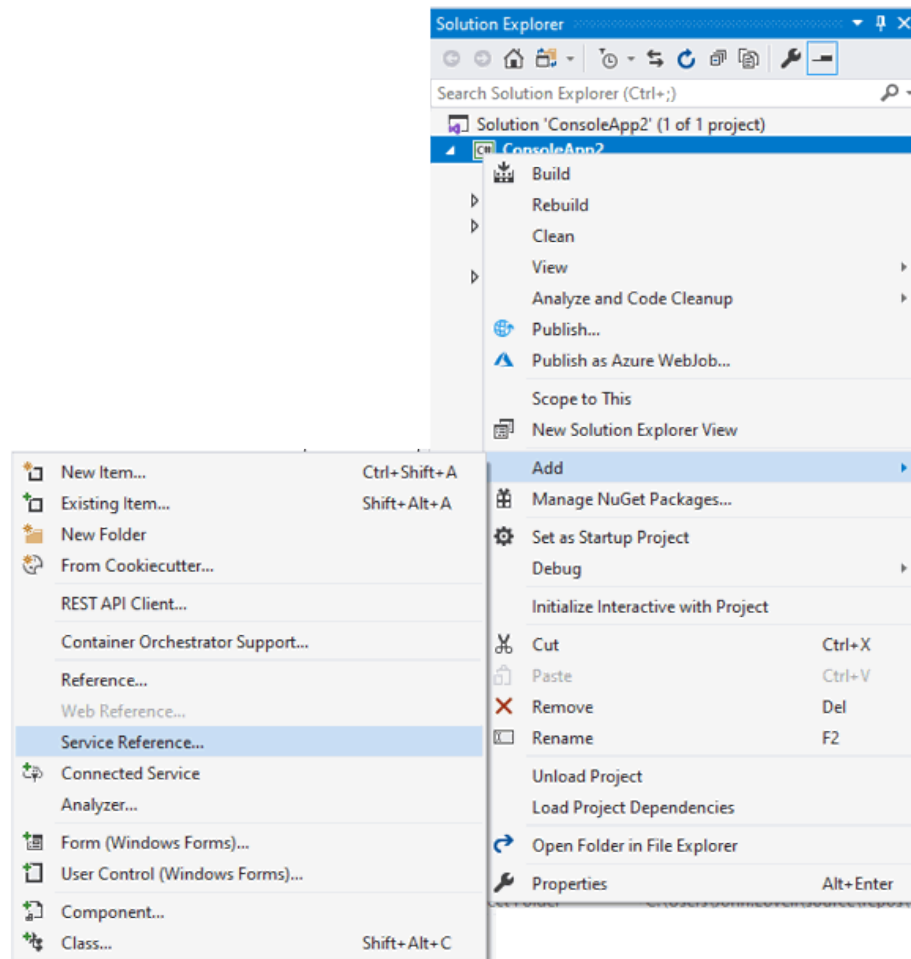
7 Click **Finish**.

Visual Studio builds the necessary reference files.

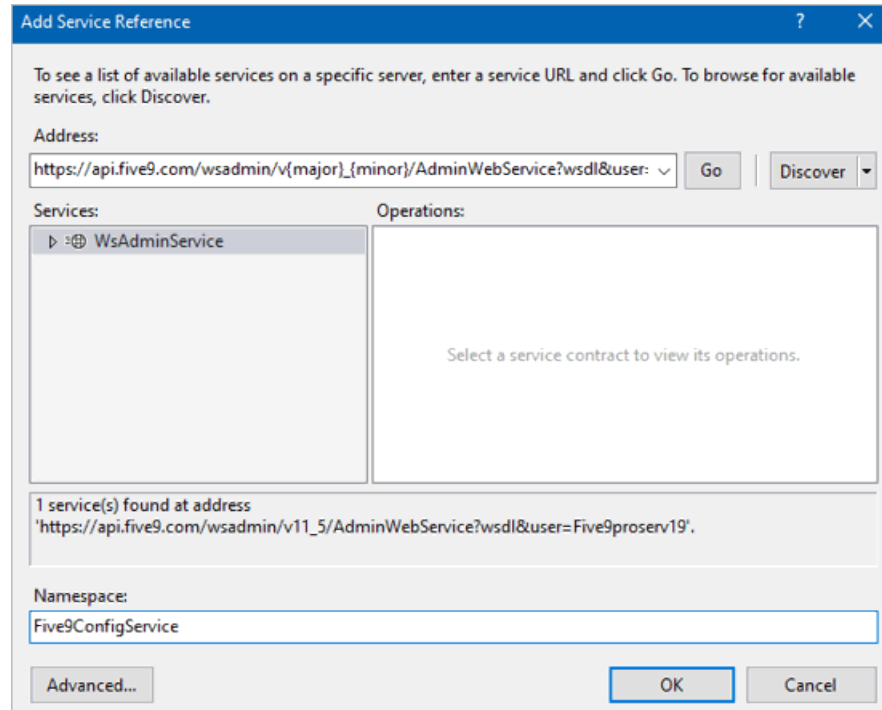
.NET Framework

This procedure provides instructions for using Visual Studio to create a .NET Framework application and add a reference to the Configuration web service.

- 1 Right-click the project in the **Solution Explorer**.



- 2 Select **Add > Service Reference**.
- 3 Enter the URI for the WSDL in the **Add Service Reference** pop-up window.



4 Click Go.

Visual Studio displays a notification that the WSDL server needs to authenticate your request.

5 Click No twice on the Discovery Credential pop-up window.

The `WsAdminService` object appears in the **Services** box upon a successful authentication. The Five9 Configuration Web Service only requires the Five9 username to identify the WSDL, which is provided in the URL. Full authentication is not required at this point.

6 Enter `Five9ConfigService` in the **Namespace field.**

The namespace is used in your code to reference the Five9 Configuration Web Service (`WsAdminService`) object.

7 Click Finish.

Visual Studio builds the necessary reference files.

Creating Code

This section describes the code you need to develop.

[Enabling HTTP Basic Authentication
Using Generated Code](#)

Enabling HTTP Basic Authentication

The Five9 Configuration web service uses Basic Authentication. The classes [AuthHeaderInserter.cs](#) and [AuthHeaderBehavior.cs](#) simplify the process of adding this header. The following code shows an example of how to make use of these classes to add the Basic Authentication header.

C#

C# Example Code

```
static void Main(string[] args){
    string _username = "";
    string _password = "";
    var binding = new BasicHttpBinding
    {
        MaxBufferSize = int.MaxValue,
        MaxBufferPoolSize = int.MaxValue,
        MaxReceivedMessageSize = int.MaxValue,
        ReaderQuotas =
        {
            MaxArrayLength = int.MaxValue,
            MaxBytesPerRead = int.MaxValue,
            MaxNameTableCharCount = int.MaxValue,
            MaxStringContentLength = int.MaxValue
        },
        Security =
        {
            Mode = BasicHttpSecurityMode.Transport,
            Transport = {ClientCredentialType = HttpCli-
entCredentialType.None}
        }
    };
    var major = 11;
    var minor = 5;
    var adminUrl = $"https://api.five9.com/wsadmin/v{major}_{minor}/Ad-
minWebService";
    var address = new EndpointAddress(adminUrl);

    var adminClient = new Five9ConfigService.WsAdminClient(binding,
address);
    var inserter = new AuthHeaderInserter
    {
        Username = _username,
        Password = _password
    };
    adminClient.Endpoint.Behaviors.Add(new AuthHead-
erBehavior(inserter));
}
```

VB.NET

VB.NET Example Code

```
Sub Main()
    Dim _username = ""
    Dim _password = ""
    ' Create proxy instance for Web service. For WsAdmiClient source,
    see Client.vb.
    Dim binding = New BasicHttpBinding With {
        .MaxBufferSize = Integer.MaxValue,
        .MaxBufferPoolSize = Integer.MaxValue,
        .MaxReceivedMessageSize = Integer.MaxValue,
        .ReaderQuotas = New Xml.XmlDictionaryReaderQuotas With {
            .MaxArrayLength = Integer.MaxValue,
            .MaxBytesPerRead = Integer.MaxValue,
            .MaxNameTableCharCount = Integer.MaxValue,
            .MaxStringContentLength = Integer.MaxValue
        },
        .Security = New BasicHttpSecurity With {
            .Mode = BasicHttpSecurityMode.Transport,
            .Transport = New HttpTransportSecurity With {
                .ClientCredentialType = HttpCli-
entCredentialType.None
            }
        }
    }

    Dim major = 11
    Dim minor = 5
    var adminUrl = $"https://api.five9.com/wsadmin/v{major}_{minor}/Ad-
minWebService";
    Dim address = New EndpointAddress(adminUrl)
    Dim service = New WsAdminClient(binding, address)
    Dim inserter = New AuthHeaderInserter With {
        .Username = _username,
        .Password = _password
    }
    service.Endpoint.EndpointBehaviors.Add(New AuthHead-
erBehavior(inserter))

End Sub
```

Using Generated Code

The code contains the following classes:

- Main wrapper class: `WsAdminClient`
- Object classes and enumerators, such as `userRoles`, `skill`, `campaignType`, and `campaignState`.

`WsAdminClient` is the main wrapper class and includes methods used to perform administrative and configurative tasks in the VCC environment. Once you have added the `Authentication` header, you can use the `adminClient` object to call these methods.

Example

The following example uses the `adminClient` object to get a list of inbound campaigns:

```
var inboundCampaigns = adminClient.getCampaigns(".*", Five9ConfigService.campaignType.INBOUND)
```

Example

The following example uses the `adminClient` object to get a list of skills:

```
var skills = adminClient.getSkills(".*");
```

Example

Some methods require objects as parameters. The following example returns a `skillInfo` object using the `adminClient` object to add a new skill named `My Test Skill`:

```
var newSkill = adminClient.createSkill(  
    new Five9ConfigService.skillInfo {  
        skill = new Five9ConfigService.skill {  
            name = "My Test Skill",  
            description = "This is an example of a test skill"  
        }  
    }  
);
```

Source Code

This section contains examples of source code to use in your implementation.

[AuthHeaderInserter.cs](#)
[AuthHeaderBehavior.cs](#)
[Program.cs](#)
[asyncAddRecordsToList.cs](#)
[AuthHeaderInserter.vb](#)
[AuthHeaderBehavior.vb](#)
[Program.vb](#)
[CreateUser.vb](#)
[Resolving Issues in the Reference.vb File](#)

AuthHeaderInserter.cs

This class is primarily used for the Admin and Supervisor web service clients, which require the Authorization HTTP header to provide security on the web service. This class implements `IClientMessageInspector` to enable an Authorization HTTP to be inserted into an HTTP message before it is sent.

C# Example Code

```
using System; using System.ServiceModel.Channels;
using System.ServiceModel.Dispatcher;
using System.Text;

namespace WebApiSample.Helpers
{
    public class AuthHeaderInserter : IClientMessageInspector
    {
        public string Username
        {
            get => _username;
            set
            {
                _username = value;
                BuildAuthString();
            }
        }
    }
}
```

```

    }

    public string Password
    {
        get => _password;
        set
        {
            _password = value;
            BuildAuthString();
        }
    }

    public string AuthHeader => _auth;

    private string _username;
    private string _password;
    private string _auth;

    private void BuildAuthString()
    {
        _auth = string.Concat("Basic ", Convert.ToBase64String(
            Encoding.UTF8.GetBytes(string.Concat(Uname, ":",
Password))));
    }
    #region IClientMessageInspector Members

    public void AfterReceiveReply(ref Message reply, object cor-
relationState)
    {
        // free httpRequest
    }

    /// <summary>
    /// Before sending an HTTP request this method inserts the Author-
    ization HTTP
    /// header using Basic security
    /// with the username:password base64 encoded.
    /// </summary>
    /// <param name="request"></param>
    /// <param name="channel"></param>
    /// <returns></returns>
    public object BeforeSendRequest(ref Message req, Sys-
tem.ServiceModel.IClientChannel ch)
    {
        HttpRequestMessageProperty httpRequest;

        object o;
        if (req.Properties.TryGetValue(HttpRequestMessageProperty.Name, out
o))
        {
            httpRequest = o as HttpRequestMessageProperty;
            if (httpRequest != null) httpRequest.Headers["Authorization"] =
AuthHeader;
        }
    }

```



```
else
{
    httpRequest = new HttpRequestMessageProperty();
    httpRequest.Headers["Authorization"] = AuthHeader;
    req.Properties.Add(HttpRequestMessageProperty.Name,
httpRequest);
}

return httpRequest;
}
#endregion
}
```

AuthHeaderBehavior.cs

This class is used to add `AuthHeaderInserter` into the endpoints behavior collection.

C# Example Code

```
using System.ServiceModel.Channels; using Sys-
tem.ServiceModel.Description;
using System.ServiceModel.Dispatcher;

namespace WebApiSample.Helpers
{
    public class AuthHeaderBehavior : IEndpointBehavior
    {
        private readonly AuthHeaderInserter _authHeaderInserter;

        public AuthHeaderBehavior(AuthHeaderInserter headerInserter)
        {
            _authHeaderInserter = headerInserter;
        }

        #region IEndpointBehavior Members

        public void AddBindingParameters(ServiceEndpoint eP, Bind-
ingParameterCollection bP)
        {
        }

        /// <summary>
```

```
/// Add the AuthHeaderInserter into the MessageInspector collection
for the endpoint.
/// </summary>
/// <param name="eP"></param>
/// <param name="clientRuntime"></param>
public void ApplyClientBehavior(ServiceEndpoint eP, ClientRuntime
clientRuntime)
{
    clientRuntime.MessageInspectors.Add(_authHead-
erInserter);
    Debug.WriteLine("AuthHeaderInserter plus Mes-
sageInspector for: " + eP.Address);
}

public void ApplyDispatchBehavior(ServiceEndpoint eP, End-
pointDispatcher ePDispatcher)
{
}

public void Validate(ServiceEndpoint eP)
{
}

#endregion
}
```

Program.cs

This is a sample class with a `Main` function that demonstrates how to initialize and authenticate to the Five9 Configuration Web Service and call the `getListsInfo` method.

C# Example

```
using System;using System.ServiceModel;
using WebApiSample.Helpers;
using WebApiSample.Five9ConfigService;

namespace WebApiSample
{
    class Program
    {
```

```
// Create instance of Web service proxy.
static void Main(string[] args)
{
    string _username = "";
    string _password = "";

    var binding = new BasicHttpBinding
    {
        MaxBufferSize = int.MaxValue,
        MaxBufferPoolSize = int.MaxValue,
        MaxReceivedMessageSize = int.MaxValue,
        ReaderQuotas =
        {
            MaxArrayLength = int.MaxValue,
            MaxBytesPerRead = int.MaxValue,
            MaxNameTableCharCount = int.MaxValue,
            MaxStringContentLength = int.MaxValue
        },
        Security =
        {
            Mode = BasicHttpSecurityMode.Transport,
            Transport = {ClientCredentialType = HttpClientCredentialType.None}
        }
    };

    var major = 11;
    var minor = 5;
    var adminUrl = $"https://api.five9.com/wsadmin/v{major}_{minor}/AdminWebService";
    var address = new EndpointAddress(adminUrl);

    var adminClient = new WsAdminClient(binding, address);
    var inserter = new AuthHeaderInserter
    {
        Username = _username,
        Password = _password
    };
    adminClient.Endpoint.Behaviors.Add(new AuthHeaderBehavior(inserter));

    listInfo[] lists = adminClient.GetListsInfo(".*");

    foreach (listInfo list in lists)
    {
        Console.WriteLine($"List {list.name} has size: {list.size}");
    }
    Console.WriteLine("\nPress Enter to close");
    Console.ReadLine();
}
```

asyncAddRecordsToList.cs

This is a sample C# class with `Main` and `asyncAddRecordsToList` functions that demonstrate how to initialize and authenticate to the Five9 Configuration Web Service and call the `asyncAddRecordsToList` API method.

C# Example Code

```
using System;using System.ServiceModel;
using ConsoleApp2.Helpers;
using ConsoleApp2.Five9ConfigService;

namespace ConsoleApp2
{
    class Program
    {
        static WsAdminClient service;
        public static void asyncAddRecordsToList()
        {
            fieldEntry number1 = new fieldEntry
            {
                columnNumber = 1,           //the first column should
have columnNumber of 1, not 0
                fieldName = "number1",
                key = true
            };
            fieldEntry last_name = new fieldEntry
            {
                columnNumber = 2,
                fieldName = "last_name",
                key = false
            };
            fieldEntry first_name = new fieldEntry
            {
                columnNumber = 3,
                fieldName = "first_name",
                key = true
            };
            listUpdateSettings updateParams = new listUpdateSettings
            (
                {
                    fieldsMapping = new fieldEntry[] { number1, last_name, first_name },
                    //skipHeaderLine = false,           //NA to method. Only
addToListCsv
                    //separator = ",",                 //NA to method. Only in
addToListCsv.
                    //reportEmail = "email@dsf.com",    //NA to method. Only in
addToList.

                    callNowMode = callNowMode.NONE,    //If set to ANY, all
```

```

records are added to ASAP
// queue.
        callNowModeSpecified = true,           //set true.
Otherwise is null.
//callNowColumnNumber = 5,           //To add only some records
to ASAP queue, a
// separate column in importData can act as a
// flag.
//callNowColumnNumberSpecified = true, //set if callNowColumnNumber
is specified.
//callTime = 333333,           //Time to call all the
records in UNIX time
// (number of milliseconds since Jan 1,1970).
//callTimeSpecified = true,           //set if callTime is spe-
cified.
//callTimeColumnNumber = 6,           //to call records at dif-
ferent times, use a
// separate column in importData to specify
// time.
//callTimeColumnNumberSpecified = true, //set if callTimeColum-
nNumber is specified.
// Source Code asyncAddRecordsToList.cs
// 14 Configuration Web Services for .NET
// Framework Programmer's Guide
        cleanListBeforeUpdate = false,
        crmAddMode = crmAddMode.ADD_NEW,
        crmAddModeSpecified = true,
        crmUpdateMode = crmUpdateMode.UPDATE_FIRST,
        crmUpdateModeSpecified = true,
        listAddMode = listAddMode.ADD_FIRST,
        listAddModeSpecified = true,
    };

    importIdentifier identifier;
try
    {
        var listName = "asdf";
        string[] resetDispositionCampaigns = { "" };
        identifier = service.asyncAddRecordsToList(
            listName,
            updateParams,
            new string[][] { new string[] { "5551234458", "Billy", "Butch" } },
            resetDispositionCampaigns
        );
    }
catch (Exception)
    {
        throw;
    }

    bool importRunning = true;
    while (importRunning == true)

```

```
        {
            importRunning = service.isImportRunning(identifier,
5);
        }
    try
    {
        listImportResult importResult = service.getListImportResult(identifier);
        Console.WriteLine
            ($"Records added to ASAP queue : {importResult.callNowQueued}");
        Console.WriteLine
            ($"Records added to CRM : {importResult.crmRecordsInserted}");
        Console.WriteLine
            ($"Records in CRM updated : {importResult.crmRecordsUpdated}");
        if (!string.IsNullOrEmpty(importResult.failureMessage))
            Console.WriteLine($"Error message : {importResult.failureMessage}");
        Console.WriteLine
            ($"Records deleted from list : {importResult.listRecordsDeleted}");
        Console.WriteLine
            ($"Records inserted in list : {importResult.listRecordsInserted}");
        Console.WriteLine
            ($"Records in list already have the same key : {importResult.uploadDuplicatesCount}");
        Console.WriteLine
            ($"Records not inserted due to error : {importResult.uploadErrorsCount}");
        if (importResult.warningsCount.Length > 0)
            Console.WriteLine("Warnings:");
        foreach (basicImportResultEntry warning in importResult.warningsCount)
        {
            Console.WriteLine($"{warning.key} : {warning.value}");
        }
    }
    catch (Exception)
    {
        throw;
    }
}

static void Main(string[] args)
{
    string _username = "";
    string _password = "";

    var binding = new BasicHttpBinding
```

```

        {
            MaxBufferSize = int.MaxValue,
            MaxBufferPoolSize = int.MaxValue,
            MaxReceivedMessageSize = int.MaxValue,
            ReaderQuotas =
            {
                MaxArrayLength = int.MaxValue,
                MaxBytesPerRead = int.MaxValue,
                MaxNameTableCharCount = int.MaxValue,
                MaxStringContentLength = int.MaxValue
            },
            Security =
            {
                Mode = BasicHttpSecurityMode.Transport,
                Transport = {ClientCredentialType = HttpCli-
entCredentialType.None}
            }
        };

        var major = 11;
        var minor = 5;
        var adminUrl = $"https://api.five9.com/wsadmin/v{major}_{minor}/Ad-
minWebService";
        var address = new EndpointAddress(adminUrl);

        var adminClient = new WsAdminClient(binding, address);
        var inserter = new AuthHeaderInserter
        {
            Username = _username,
            Password = _password
        };
        adminClient.Endpoint.Behaviors.Add(new AuthHead-
erBehavior(inserter));

        asyncAddRecordsToList();

        Console.Write("\nPress Enter to close");
        Console.ReadLine();
    }
}

```

AuthHeaderInserter.vb

This class is primarily used for the Admin and Supervisor web service clients because those web services require the Authorization HTTP header to provide security on the

web service. This class implements `IClientMessageInspector` so that an Authorization HTTP header may be inserted into an HTTP message before it is sent.

VB Example Code

```
Imports System.ServiceModelImports System.ServiceModel.Channels
Imports System.ServiceModel.Dispatcher
Imports System.Text

Public Class AuthHeaderInserter
    Implements IClientMessageInspector

    Public Property Username As String
    Get
    Return _username
    End Get
    Set(ByVal value As String)
        _username = value
        BuildAuthString()
    End Set
    End Property

    Public Property Password As String
    Get
    Return _password
    End Get
    Set(ByVal value As String)
        _password = value
        BuildAuthString()
    End Set
    End Property

    Public ReadOnly Property AuthHeader As String
    Get
    Return _auth
    End Get
    End Property

    Private _username As String
    Private _password As String
    Private _auth As String

    Private Sub BuildAuthString()
        _auth = String.Concat(
            "Basic ",
            Convert.ToBase64String(
                Encoding.UTF8.GetBytes(String.Concat(Username, ":",
                    Password))
            )
        )
    End Sub
```



```
Private Function IClientMessageInspector_BeforeSendRequest (ByRef
request As Message,
channel As IClientChannel)
    As Object Implements IClientMessageInspector.BeforeSendRequest
Dim httpRequest As HttpRequestMessageProperty
Dim o As Object

If request.Properties.TryGetValue (HttpRequestMessageProperty.Name,
o) Then
    httpRequest = TryCast(o, HttpRequestMessageProperty)
If httpRequest IsNot Nothing Then httpRequest.Headers("Author-
ization") = AuthHeader
Else
    httpRequest = New HttpRequestMessageProperty()
    httpRequest.Headers("Authorization") = AuthHeader
    request.Properties.Add (HttpRequestMessageProperty.Name,
httpRequest)
End If

Return httpRequest
End Function

Private Sub IClientMessageInspector_AfterReceiveReply (ByRef reply As
Message,
correlationState As Object) Implements ICli-
entMessageInspector.AfterReceiveReply
End Sub
End Class
```

AuthHeaderBehavior.vb

This class is used to add the `AuthHeaderInserter` into the endpoints behavior collection.

VB Example Code

```
Imports System.ServiceModel.ChannelsImports Sys-
tem.ServiceModel.Description
Imports System.ServiceModel.Dispatcher
Public Class AuthHeaderBehavior
Implements IEndpointBehavior

Private ReadOnly _authHeaderInserter As AuthHeaderInserter

Public Sub New (ByVal headerInserter As AuthHeaderInserter)
```

```
        _authHeaderInserter = headerInserter
    End Sub

    Private Sub IEndpointBehavior_Validate(
        endpoint As ServiceEndpoint
    ) Implements IEndpointBehavior.Validate
    End Sub

    Private Sub IEndpointBehavior_AddBindingParameters(
        endpoint As ServiceEndpoint,
        bindingParameters As BindingParameterCollection
    ) Implements IEndpointBehavior.AddBindingParameters
    End Sub

    Private Sub IEndpointBehavior_ApplyDispatchBehavior(
        endpoint As ServiceEndpoint,
        endpointDispatcher As EndpointDispatcher
    ) Implements IEndpointBehavior.ApplyDispatchBehavior
    End Sub

    Private Sub IEndpointBehavior_ApplyClientBehavior(
        endpoint As ServiceEndpoint,
        clientRuntime As ClientRuntime
    ) Implements IEndpointBehavior.ApplyClientBehavior
        clientRuntime.MessageInspectors.Add(_authHeaderInserter)
    End Sub
End Class
```

Program.vb

This is a sample class with a `Main` function that demonstrates how to initialize and authenticate to the Five9 Configuration Web Service and call the `getListsInfo` method.

VB Example Code

```
getListsInfo method.
Imports VBConsoleAppl.Five9ConfigService
Imports System.ServiceModel

Module Program
```

```

Sub GetUsers(ByRef service As WsAdminClient, ByVal pattern
As String)
    ' Call operation and get result
    Dim result() As userInfo = service.getUsersInfo(pattern)
    If result.Length > 0 Then
        Dim i As Integer = 0
        For Each user As userInfo In result
            Console.WriteLine(i.ToString + ". " + user-
.generalInfo.userName)
            i = i + 1
        Next
    Else
        Console.WriteLine("The result for pattern " + pattern +
" is empty. ")
    End If
End Sub

Sub Main()
    Dim _username = ""
    Dim _password = ""
    ' Create proxy instance for Web service. For WsAdmiClient source,
    see Client.vb.
    Dim binding = New BasicHttpBinding With {
        .MaxBufferSize = Integer.MaxValue,
        .MaxBufferPoolSize = Integer.MaxValue,
        .MaxReceivedMessageSize = Integer.MaxValue,
        .ReaderQuotas = New Xml.XmlDictionaryReaderQuotas With {
            .MaxArrayLength = Integer.MaxValue,
            .MaxBytesPerRead = Integer.MaxValue,
            .MaxNameTableCharCount = Integer.MaxValue,
            .MaxStringContentLength = Integer.MaxValue
        },
        .Security = New BasicHttpSecurity With {
            .Mode = BasicHttpSecurityMode.Transport,
            .Transport = New HttpTransportSecurity With {
                .ClientCredentialType = HttpCli-
entCredentialType.None
            }
        }
    }

    Dim major = 11
    Dim minor = 5
    Dim adminUrl = $"https://api.five9.com/wsadmin/v{major}_{minor}/Ad-
minWebService"
    Dim address = New EndpointAddress(adminUrl)

    Dim service = New WsAdminClient(binding, address)
    Dim inserter = New AuthHeaderInserter With {
        .Username = _username,
        .Password = _password
    }

```

```
        service.Endpoint.EndpointBehaviors.Add(New AuthHeaderBehavior(inserter))

    ' Fetch all users for domain.
    GetUsers(service, ".*")
    ' Wait until user presses Enter.
    Console.WriteLine("Press Enter to exit")
    Console.ReadLine()
End Sub
End Module
```

CreateUser.vb

This is a sample VB.NET module with `Main` and `CreateUser` functions that demonstrate how to initialize and authenticate to the Five9 Configuration Web Service and call the `createUser` API method.

Note: When setting any parameter that is optional, you need to include the corresponding specified parameter. For example, if you want to allow the user to change password, include both the optional `.canChangePassword = true`, and specified parameter `.canChangePasswordSpecified = true`.

VB Example Code

```
Imports VBConsoleAppl.Five9ConfigServiceImports System.Web.Services.Protocols
Imports System.ServiceModel

Module CreateUser
Sub CreateUser(ByRef service As WsAdminClient)
    ' Create argument instance to be passed to service operation.
    ' Define General User Information, such as Name, Username, Password.
    Dim NewUserGeneral As New userGeneralInfo With {
        .fullName = "John Smith",
        .userName = "Smithy",
        .EMail = "something@somedomain.com",
        .canChangePassword = True,
        .mustChangePassword = True,
        .password = "some_password"
    }
End Sub
End Module
```

```

' To give Agent role, add 1 or more permissions:
Dim NewPermCallForwarding As New agentPermission With {
    .type = agentPermissionType.CallForwarding,
    .typeSpecified = True,
    .value = True
}
Dim NewPermCreateConference As New agentPermission With {
    .type = agentPermissionType.CreateConferenceWithAgents,
    .typeSpecified = True,
    .value = True
}
Dim myAgent As New agentRole With {
    .permissions = {NewPermCallForwarding, NewPermCreateConference},
    .alwaysRecorded = False,
    .attachVmToEmail = False
}
' To give Admin role, add 1 or more permissions:
Dim admperl As New adminPermission With {
    .type = adminPermissionType.FullPermissions,
    .typeSpecified = True,
    .value = True
}
' To give Supervisor role, add 1 or more permissions:
Dim superP As New supervisorPermission With {
    .type = supervisorPermissionType.Agents,
    .typeSpecified = True,
    .value = True
}
' Create userRoles object with agent, admin, and supervisor:
Dim NewRoles As New userRoles With {
    .agent = myAgent,
    .admin = {admperr1},
    .supervisor = {superP}
}
' specify 1 or more user-skill relations:
Dim NewSkill As New userSkill With {
    .skillName = "some_skill",
    .userName = "Smithy",

```

```

        .level = 1
    }

    ' Parameters for createUser are generalInfo, roles, and skills:
    Dim NewUser As New userInfo With {
        .generalInfo = NewUserGeneral,
        .roles = NewRoles,
        .skills = {NewSkill}
    }

    ' Execute and check for errors:
    Try
        Dim userInfo = service.createUser(NewUser)
        Console.WriteLine(userInfo.generalInfo.fullName)
    Catch myException As SoapException
        Console.WriteLine(myException.Message)
    End Try
End Sub

Sub Main()
    Dim _username = ""
    Dim _password = ""
    ' Create proxy instance for Web service. For WsAdmiClient source,
    see Client.vb.
    Dim binding = New BasicHttpBinding With {
        .MaxBufferSize = Integer.MaxValue,
        .MaxBufferPoolSize = Integer.MaxValue,
        .MaxReceivedMessageSize = Integer.MaxValue,
        .ReaderQuotas = New Xml.XmlDictionaryReaderQuotas With {
            .MaxArrayLength = Integer.MaxValue,
            .MaxBytesPerRead = Integer.MaxValue,
            .MaxNameTableCharCount = Integer.MaxValue,
            .MaxStringContentLength = Integer.MaxValue
        },
        .Security = New BasicHttpSecurity With {
            .Mode = BasicHttpSecurityMode.Transport,
            .Transport = New HttpTransportSecurity With {
                .ClientCredentialType = HttpCli-
entCredentialType.None
            }
        }
    }

    Dim major = 11
    Dim minor = 5
    Dim adminUrl = $"https://api.five9.com/wsadmin/v{major}_{minor}/Ad-
minWebService"
    Dim address = New EndpointAddress(adminUrl)

    Dim service = New WsAdminClient(binding, address)
    Dim inserter = New AuthHeaderInserter With {

```

```
        .Username = _username,  
        .Password = _password  
    }  
  
    service.Endpoint.EndpointBehaviors.Add(New AuthHeaderBehavior(inserter))  
  
    CreateUser(service)  
    ' Wait until user presses Enter.  
    Console.WriteLine("Press Enter to exit.")  
    Console.ReadLine()  
End Sub  
End Module
```

Resolving Issues in the Reference.vb File

Due to the case-insensitive nature of Visual Basic, the Five9 Configuration Web Service WSDL introduces an error in the generated `Reference.vb` file. This error is specific to the `system` property of `contactField`. To resolve this issue, rename this property to `isSystem` by following these steps:

- 1 Open the `Reference.vb` file located in the `\Connected Services\Five9ConfigService` folder.
 - In the Solution Explorer, click on the Show All Files icon at the top.
 - Expand `Five9ConfigService`.
 - Expand `Reference.svcmap`.
 - Double-click `Reference.vb`.
- 2 Search for `Public Property system() As Boolean` in the `Reference.vb` file (there should only be one result) and make the following changes:
 - a Rename the `contactField.system` property definition to `contactField.isSystem`.
 - b Replace `Me.RaisePropertyChanged("system")` with

Me.RaisePropertyChanged("isSystem") in the following lines:

```
Public Property system() As Boolean
    Get
        Return Me.systemField
    End Get
    Set
        Me.systemField = value
        Me.RaisePropertyChanged("system")
    End Set
End Property
```

The resulting lines should look like the following:

```
Public Property isSystem() As Boolean
    Get
        Return Me.systemField
    End Get
    Set
        Me.systemField = value
        Me.RaisePropertyChanged("isSystem")
    End Set
End Property
```